

完成情况

- Gemini Max 深度图正常发布，约 30 Hz
- 点云正常发布，约 30 Hz
- RViz 能显示点云
- RViz 右下角 2~3 fps 只是远程界面的渲染速度，不是相机实际帧率

下面是一份完整总结。

Jetson Orin Nano Super 4GB + Orbbec Gemini Max + ROS 2 Humble 深度相机配置总结

1. 项目目标

本次工作的目标是在 Jetson Orin Nano Super 4GB 上，通过 ROS 2 驱动奥比中光 Gemini Max 深度相机，并完成以下功能：

1. 识别相机设备
2. 获取深度图
3. 在 ROS 2 中发布深度图话题
4. 根据深度图生成三维点云
5. 使用 `rqt_image_view` 查看深度图
6. 使用 RViz2 查看点云
7. 测量深度图和点云的实际发布频率

最终以上目标均已完成。

2. 硬件与软件环境

2.1 硬件环境

- 计算平台：Jetson Orin Nano Super 4GB
- 相机：奥比中光 Orbbec Gemini Max
- 相机内部设备名称：SV1301S_U3
- USB PID：0x0614
- 相机固件：RD3013
- 当前连接模式：USB 2.0
- 主机架构：aarch64

2.2 软件环境

- 操作系统：Ubuntu 22.04.5 LTS
- Ubuntu 代号：jammy
- JetPack：6.2

- L4T: R36.4.7
- ROS 2: Humble
- ROS 2 工作空间:



/home/jetson/orbbec_ros2_ws

- ROS 2 驱动源码:



/home/jetson/orbbec_ros2_ws/src/OrbbecSDK_ROS2

- 使用的驱动分支:



main

3. 为什么原教程不能直接照抄

亚博教程使用的是:



Ubuntu 20.04 + ROS 2 Foxy

而实际设备使用的是:



Ubuntu 22.04 + ROS 2 Humble

ROS 2 发行版与 Ubuntu 版本通常存在对应关系:

Ubuntu	推荐 ROS 2
Ubuntu 20.04	Foxy、Galactic
Ubuntu 22.04	Humble
Ubuntu 24.04	Jazzy

因此, 教程中的:



ros-foxy-image-transport

在当前系统中应对应为:



ros-humble-image-transport

不能简单安装 Foxy, 因为 Foxy 并不是 Ubuntu 22.04 的标准配套版本。强行混装不同 ROS 2 版本, 容易造成软件源、动态库和 Python 版本冲突。

4. “Python 虚拟环境里的驱动”是什么

之前在下面的 Python 虚拟环境中配置过相机：



```
/home/jetson/pyorbbecsdk-main/gemini
```

激活方式为：



```
cd ~/pyorbbecsdk-main
source gemini/bin/activate
```

这里需要区分三个概念。

4.1 USB 和 udev 层

负责让 Linux 系统发现相机，并允许普通用户访问 USB 设备。

4.2 Orbbec SDK 层

负责和相机通信，读取深度帧、彩色帧、红外帧等原始数据。

4.3 ROS 2 驱动层

负责调用 Orbbec SDK，再把相机数据转换成 ROS 2 话题，例如：



```
/camera/depth/image_raw
/camera/depth/points
```

Python 虚拟环境主要隔离 Python 包。它不是一个真正的虚拟机，也不会把 USB 驱动完整地“装在里面”。

Python SDK 能运行，只能说明：



```
Linux能访问相机
→ Orbbec SDK能够识别相机
→ Python程序能够读取数据
```

但这不等于 ROS 2 驱动已经安装。ROS 2 驱动仍然需要单独下载、编译和加载。

正常运行 ROS 2 相机节点时，一般不需要进入 Python 虚拟环境。

5. Python SDK 测试阶段遇到的问题

5.1 相机检测成功

使用 `hello_orbbec.py` 成功识别到了相机，说明以下部分正常：

- USB连接正常
- Linux能够发现设备
- 相机固件能够响应

- Python SDK可以和相机通信
- 基础设备权限基本正常

这一步很重要，因为它把问题范围缩小到了“显示环境或 ROS 驱动”，而不是相机硬件本身。

5.2 `depth_viewer.py` 无法显示窗口

当时安装的是：



```
opencv-python-headless
```

`headless` 表示无图形界面版本。它适合服务器进行图像计算，但通常不包含 `cv2.imshow()` 所需的 GUI 后端。

因此程序虽然可能已经获得深度帧，却无法弹出显示窗口。

解决方法是卸载：



```
pip uninstall opencv-python-headless
```

让系统中支持图形界面的 OpenCV 生效。

处理后，`depth_viewer.py` 成功显示深度图。这进一步证明相机和底层 SDK 工作正常。

6. 为什么选择 ROS 2 驱动的 `main` 分支

OrbbecSDK_ROS2 仓库存在不同驱动分支，不能仅根据“哪个分支更新”来选择。

经过设备信息、仓库说明和技术客服确认：

- Gemini Max 属于旧 OpenNI 协议设备
- 当前相机应该使用 SDK v1 兼容驱动
- 对应仓库的 `main` 分支
- `v2-main` 主要面向新的 Orbbec SDK v2 设备

因此最终使用：



```
OrbbecSDK_ROS2/main
```

而没有使用：



```
v2-main
```

这一步的核心原则是：

驱动版本必须和相机内部使用的通信协议匹配，而不是单纯选择版本号最新的分支。

如果协议不匹配，可能出现相机找不到、启动后退出、配置格式不支持或没有图像等问题。

7. ROS 2 工作空间是什么

创建的工作空间为：

```
● ● ●
~/orbbec_ros2_ws
```

典型结构如下：

```
● ● ●
orbbec_ros2_ws/
├─ src/      # 驱动源代码
├─ build/    # 编译过程中的中间文件
├─ install/  # 编译完成后生成的可运行文件
└─ log/      # 编译日志
```

源码放在：

```
● ● ●
~/orbbec_ros2_ws/src/OrbbecSDK_ROS2
```

驱动仓库中包含三个 ROS 2 功能包：

```
● ● ●
orbbec_camera
orbbec_camera_msgs
orbbec_description
```

它们的大致作用是：

- `orbbec_camera`：相机节点、启动文件和主要驱动逻辑
- `orbbec_camera_msgs`：Orbbec 自定义 ROS 消息
- `orbbec_description`：相机模型以及相关坐标系描述

8. rosdep 是什么

`rosdep` 是 ROS 的系统依赖管理工具。

ROS 功能包会在 `package.xml` 中声明依赖名称，例如：

```
● ● ●
rclcpp
sensor_msgs
image_transport
```

但不同 Ubuntu 和 ROS 版本对应的系统安装包名称可能不同。`rosdep` 的作用是：

```
● ● ●
读取 package.xml
→ 判断当前 Ubuntu 和 ROS 版本
→ 找到对应的 apt 软件包
→ 自动安装缺少的依赖
```

使用的命令为：



```
cd ~/orbbec_ros2_ws
rosdep install --from-paths src --ignore-src -r -y
```

各参数含义：

- `--from-paths src`：扫描 `src` 目录中的 ROS 包
- `--ignore-src`：源码中已经存在的包不重复安装
- `-r`：某个依赖失败时继续处理其他依赖
- `-y`：安装过程中自动确认

9. rosdep 未初始化和网络错误

第一次运行时提示 rosdep 尚未初始化。

rosdep 首次使用通常需要：



```
sudo rosdep init
rosdep update
```

两条命令作用不同：

- `sudo rosdep init`：创建 rosdep 的系统级数据源配置
- `rosdep update`：下载依赖名称与系统软件包之间的映射数据

`rosdep update` 访问 GitHub 上的规则文件，因此可能因为网络不稳定、DNS、代理或连接超时而报错。

这类错误不一定代表 rosdep 完全不可用。如果本地已经有部分缓存，依赖安装仍有可能成功。

最终得到：



```
All required rosdeps installed successfully
```

说明驱动声明的系统依赖已经满足，因此没有必要继续反复处理之前的网络警告。

需要注意：



```
sudo rosdep init
```

通常只需要执行一次，而：



```
rosdep update
```

应使用普通用户运行，一般不加 `sudo`。

10. 编译 ROS 2 驱动

在工作空间根目录执行：



```
cd ~/orbbec_ros2_ws
colcon build
```

colcon 是 ROS 2 常用的工作空间构建工具。它会：

1. 查找 **src** 中的 ROS 2 功能包
2. 分析包之间的依赖顺序
3. 调用 CMake 或其他构建系统
4. 编译 C++ 程序
5. 将结果安装到 **install** 目录

最终结果为：



```
Summary: 3 packages finished
```

这意味着三个功能包均已成功编译。编译成功只能证明代码和依赖没有构建错误，还需要启动相机才能验证运行时是否正常。

11. 为什么每个终端都要执行 **source**

运行 ROS 2 驱动前，需要执行：



```
source /opt/ros/humble/setup.bash
source ~/orbbec_ros2_ws/install/setup.bash
```

第一条加载系统安装的 ROS 2 Humble：



```
source /opt/ros/humble/setup.bash
```

它让终端知道 **ros2** 命令、标准消息以及 ROS 2 基础库在哪里。

第二条加载自己编译的 Orbbec 工作空间：



```
source ~/orbbec_ros2_ws/install/setup.bash
```

它让 ROS 2 找到刚刚编译的：



```
orbbec_camera
orbbec_camera_msgs
orbbec_description
```

可以把它理解为两层环境：



```
ROS 2 Humble基础环境
↓
Orbbec相机驱动工作空间
```

每打开一个新终端，默认都不会继承另一个终端执行过的 **source**，所以通常需要重新执行。

12. udev 规则的作用

Linux 默认会对 USB 设备设置访问权限。没有正确的 udev 规则时，普通用户可能无法打开相机，并出现类似问题：



```
Permission denied
No device found
Failed to open device
```

安装 Orbbec 提供的 udev 规则后，系统可以根据相机的 USB VID/PID 自动设置设备权限。

udev 规则不是相机算法驱动，它主要解决：



```
设备插入后叫什么
普通用户是否有权访问
插拔后如何自动应用权限
```

当前相机已经被普通用户正常启动，说明设备访问链路已经正常。

13. 第一次启动 ROS 2 相机失败：默认 Y14 不受支持

使用通用启动文件：



```
ros2 launch orbbec_camera ob_camera.launch.py
```

驱动默认尝试使用某种深度格式，其中默认的 **Y14** 不被这台 Gemini Max 当前配置支持。

这不是 ROS 2 安装失败，也不是相机损坏，而是：



```
启动文件请求的深度格式
#
相机固件支持的深度格式
```

相机支持 **Y12**，因此需要显式指定：



```
depth_format:=Y12
```

最终可用的启动命令为：



```
ros2 launch orbbec_camera ob_camera.launch.py \
  enable_color:=false \
  enable_ir:=false \
  enable_depth:=true \
  depth_width:=640 \
  depth_height:=400 \
  depth_fps:=30 \
  depth_format:=Y12 \
  enable_point_cloud:=true
```

参数含义：

参数	作用
<code>enable_color:=false</code>	暂时关闭彩色相机
<code>enable_ir:=false</code>	暂时关闭红外图像
<code>enable_depth:=true</code>	开启深度图
<code>depth_width:=640</code>	深度图宽度为640像素
<code>depth_height:=400</code>	深度图高度为400像素
<code>depth_fps:=30</code>	请求每秒30帧
<code>depth_format:=Y12</code>	使用相机支持的深度数据格式
<code>enable_point_cloud:=true</code>	根据深度图生成点云

先关闭彩色和红外的原因是：当前相机显示为 `USB 2.0` 连接，同时开启多个图像流可能造成 USB 带宽不足。先完成深度和点云的最小闭环，更容易定位问题。

14. 相机成功启动的证据

启动日志出现了以下关键信息：

● ● ●

Device SV1301S_U3 connected
stream depth is enabled
width: 640, height: 400, fps: 30, format: OB_FORMAT_Y12
Publishing static transform

这些信息分别表示：

- 驱动找到了 Gemini Max
- 深度数据流已开启
- 实际配置是 `640x400@30 FPS`
- 深度格式为 `Y12`
- 相机坐标系之间的静态变换已经发布

对应版本为：

● ● ●

Wrapper version: 1.5.22
SDK version: 1.10.37

15. ROS 2 中已经发布的话题

使用：

● ● ●

ros2 topic list

确认存在：

```
● ● ●  
/camera/depth/camera_info  
/camera/depth/image_raw  
/camera/depth/points  
/camera/depth_filter_status
```

各话题作用如下。

/camera/depth/image_raw

原始深度图。每个像素表示相机在该方向测得的距离值。

它不是普通灰度照片。显示工具会将距离范围映射为灰度或伪彩色，方便人眼观察。

/camera/depth/camera_info

相机标定参数，主要包括：

- 焦距 `fx`、`fy`
- 光心 `cx`、`cy`
- 畸变参数
- 图像宽度和高度

生成准确点云时必须使用这些内参。

/camera/depth/points

由深度图和相机内参计算出的三维点云，消息类型通常是：

```
● ● ●  
sensor_msgs/msg/PointCloud2
```

/camera/depth_filter_status

与深度滤波器状态有关的信息，不是主要图像数据。

16. 深度图是怎样变成点云的

深度图中的一个像素可以表示为：

```
● ● ●  
(u, v, depth)
```

其中：

- `u`：像素横坐标
- `v`：像素纵坐标
- `depth`：该方向测得的深度

利用相机内参，可以近似投影到三维空间：



```
Z = depth
X = (u - cx) * Z / fx
Y = (v - cy) * Z / fy
```

于是每个有效深度像素都会变成一个三维点：



```
(X, Y, Z)
```

大量三维点组合起来就是点云。

因此处理流程是：



```
真实环境
→ 相机测量距离
→ 深度图
→ 结合相机内参进行反投影
→ 三维点云
→ RViz显示
```

17. 查看深度图

运行：



```
ros2 run rqt_image_view rqt_image_view
```

在左上角选择：



```
/camera/depth/image_raw
```

成功看到深度图说明：

- 相机持续采集深度数据
- ROS 2 相机节点正常运行
- 图像消息成功发布
- ROS 2 订阅端能够收到数据
- X11 图形转发可以显示二维窗口

18. 查看点云

运行：



```
rviz2
```

RViz 中设置：



Fixed Frame: camera_depth_optical_frame

添加:



PointCloud2

选择话题:



/camera/depth/points

推荐显示参数:



Style: Points
Size (Pixels): 1
Decay Time: 0
Queue Size: 1

Fixed Frame 是 RViz 用来统一表示所有数据的参考坐标系。使用深度相机光学坐标系，可以让点云直接在自身坐标系中显示，减少不必要的坐标变换问题。

点云初看起来很小，是因为 RViz 观察视角距离约为10米，并不是点云只生成了一小块。可以点击 **Focus Camera**，再使用鼠标滚轮缩放。

19. 深度图和点云的真实帧率

测量深度图:



```
ros2 topic hz /camera/depth/image_raw
```

得到约:



30.06 Hz

测量点云:



```
ros2 topic hz /camera/depth/points
```

得到约:



30.10 Hz

Hz 表示每秒发生的次数:



30 Hz ≈ 每秒30帧

因此已经客观验证：

数据	实际发布频率	状态
深度图	约30 Hz	正常
点云	约30 Hz	正常
RViz画面	约2 ~ 3 fps	显示较慢

刚运行 `ros2 topic hz` 时出现：

```
does not appear to be published yet
```

只是测量程序在订阅建立前还没有收到第一条消息。后续持续测得约30 Hz，说明它不是实际故障。

20. 为什么 RViz 右下角只有 2 ~ 3 fps

RViz 右下角的：

```
2 fps
```

表示 RViz 图形界面的渲染频率，不代表 `/camera/depth/points` 的发布频率。

每帧深度图的理论像素数为：

```
640 × 400 = 256000
```

如果每个有效像素都生成一个三维点，那么每秒最多需要处理：

```
256000 × 30 = 7680000 个点
```

当前又存在以下条件：

- Jetson 只有4GB内存
- RViz需要进行三维渲染
- 画面通过 MobaXterm X11 转发到电脑
- X11需要传输大量图形绘制指令
- 远程环境可能没有充分使用Jetson GPU

因此数据仍然以约30 Hz发布，但 RViz 只能以约2 ~ 3 fps绘制。

终端中的：

```
Message Filter dropping message  
discarding message because the queue is full
```

表示：



点云以约30帧进入RViz
→ RViz来不及处理
→ 等待队列被装满
→ RViz丢弃旧消息
→ 优先显示较新的点云

这不表示相机或 ROS 驱动故障，而是显示端性能不足。

Stereo is NOT SUPPORTED 只表示当前 OpenGL 环境不支持立体视觉显示模式。普通 RViz 点云显示不依赖该功能，可以忽略。

21. 建议的 RViz 优化

可以将 RViz 的全局 **Frame Rate** 从 **30** 改为：



5

这只降低 RViz 的目标刷新率，不会修改相机和点云话题的30 Hz发布频率。

同时建议：



PointCloud2 Style: Points
Size (Pixels): 1
Decay Time: 0
Queue Size: 1

还可以：

- 查看点云时关闭 **rqt_image_view**
- 不要同时开启多个 RViz 窗口
- 不要将 **Queue Size** 调得很大
- 优先在 Jetson 本机显示器上运行 RViz
- 远程使用时降低点云分辨率或帧率

增大队列不能提高性能，只会积压旧点云，最终让画面产生更严重的延迟。

22. 当前已经完成的完整链路



```
Gemini Max硬件
↓
Linux USB识别和设备权限
↓
Orbbec SDK读取深度数据
↓
ROS 2 Orbbec驱动
↓
/camera/depth/image_raw, 约30 Hz
↓
相机内参反投影
↓
/camera/depth/points, 约30 Hz
```

↓
RViz2显示点云

最终结论：

Gemini Max 已经在 Jetson Orin Nano Super 4GB 的 Ubuntu 22.04 和 ROS 2 Humble 环境中正常运行。深度图与点云均以约30 Hz 发布。当前2 ~ 3 fps只出现在MobaXterm远程RViz渲染端，不是相机采集或ROS 2点云发布异常。

23. 以后启动相机的标准步骤

终端一：启动相机

```
● ● ●
source /opt/ros/humble/setup.bash
source ~/orbbec_ros2_ws/install/setup.bash

ros2 launch orbbec_camera ob_camera.launch.py \
  enable_color:=false \
  enable_ir:=false \
  enable_depth:=true \
  depth_width:=640 \
  depth_height:=400 \
  depth_fps:=30 \
  depth_format:=Y12 \
  enable_point_cloud:=true
```

这个终端要一直保持运行。按 **Ctrl+C** 会停止相机节点。

终端二：检查话题

```
● ● ●
source /opt/ros/humble/setup.bash
source ~/orbbec_ros2_ws/install/setup.bash
ros2 topic list
```

查看深度图

```
● ● ●
ros2 run rqt_image_view rqt_image_view
```

选择：

```
● ● ●
/camera/depth/image_raw
```

查看点云

```
● ● ●
rviz2
```

配置：



Fixed Frame: camera_depth_optical_frame
PointCloud2 Topic: /camera/depth/points

检查帧率



```
ros2 topic hz /camera/depth/image_raw
ros2 topic hz /camera/depth/points
```

测量完成后按 **Ctrl+C**，只会结束测量命令，不会关闭相机节点。

24. 常见问题快速判断

现象	可能原因	判断方法
<code>ros2: command not found</code>	没加载ROS环境	source Humble环境
找不到 <code>orbdec_camera</code>	没加载工作空间	source工作空间的setup.bash
<code>No device found</code>	USB、权限或设备占用	检查USB和udev
Y14格式错误	相机不支持默认格式	设置 <code>depth_format:=Y12</code>
没有点云话题	没开启点云	设置 <code>enable_point_cloud:=true</code>
深度图正常但点云不显示	RViz话题或Fixed Frame错误	检查PointCloud2和坐标系
RViz只有2~3 fps	远程渲染性能不足	用 <code>ros2 topic hz</code> 测真实数据
<code>queue is full</code>	RViz处理速度低于消息速度	Queue Size设为1并降低显示刷新率
<code>Stereo is NOT SUPPORTED</code>	OpenGL不支持立体显示	普通点云显示可忽略
rosdep下载超时	网络访问规则服务器不稳定	重试或使用已有缓存

25. 接下来值得学习的方向

25.1 深度图距离读取

编写 ROS 2 节点订阅：



/camera/depth/image_raw

读取图像中心像素的深度，从而获得相机正前方障碍物距离。

25.2 点云处理

使用 PCL 或 Open3D 完成：

- 体素降采样
- 离群点去除
- 平面分割
- 地面去除
- 聚类
- 障碍物检测
- 三维尺寸估计

25.3 坐标系 TF

学习以下坐标系概念：

- 相机坐标系
- 光学坐标系
- 机器人机体坐标系 `base_link`
- 世界坐标系 `map`
- 里程计坐标系 `odom`

具身智能系统不仅要“看见一个点”，还要知道这个点相对于机器人和世界位于哪里。

25.4 彩色与深度对齐

后续可以开启彩色图，研究：

- 彩色图与深度图分辨率差异
- 相机内参和外参
- 深度到彩色图对齐
- 彩色点云
- RGB-D目标检测

但由于当前相机工作在 `USB 2.0`，开启彩色、红外、深度和点云前，应先评估 USB 带宽以及 Jetson 4GB 的内存压力。

25.5 从“看到点云”到“机器人理解环境”

后续完整技术路线可以是：

深度相机

→ 深度图和点云

→ 点云滤波

→ 地面分割

→ 障碍物检测

→ 目标识别

→ 三维定位

→ TF坐标转换

→ 地图与导航

→ 机械臂或移动底盘执行动作

现在完成的是这条路线中非常关键的第一步：**稳定获得可信的三维视觉数据，并能够证明数据的实际发布频率。**

26. Gemini Max 红外图配置与问题总结

在完成深度图和点云后，继续验证了 Gemini Max 的红外成像功能。

本阶段最终确认：

- ROS 2 驱动能够识别 Gemini Max 的红外传感器
- 红外数据流成功开启
- 实际支持的红外格式为 Y10
- 红外分辨率为 640 × 400
- 配置帧率为 30 FPS
- ROS 2 成功发布 /camera/ir/image_raw
- rqt_image_view 能够订阅红外图像

27. 查看驱动支持的红外参数

使用以下命令查看通用启动文件支持的全部参数：

```
source /opt/ros/humble/setup.bash
source ~/orbbec_ros2_ws/install/setup.bash

ros2 launch orbbec_camera ob_camera.launch.py --show-args
```

这条命令不会启动相机，只会读取启动文件并列出可以传入的参数。

与红外相关的参数如下：

```
ir_width
ir_height
ir_fps
ir_format
enable_ir
flip_ir
ir_qos
ir_camera_info_qos
enable_ir_auto_exposure
```

启动文件给出的默认配置是：

```
ir_width: 640
ir_height: 400
ir_fps: 30
ir_format: Y8
enable_ir: true
```

各参数含义如下：

参数	作用
ir_width	红外图像宽度

参数	作用
<code>ir_height</code>	红外图像高度
<code>ir_fps</code>	红外图像目标帧率
<code>ir_format</code>	红外原始像素格式
<code>enable_ir</code>	是否开启红外数据流
<code>flip_ir</code>	是否翻转红外图像
<code>ir_qos</code>	红外图像话题的ROS 2通信策略
<code>ir_camera_info_qos</code>	红外相机标定信息的通信策略
<code>enable_ir_auto_exposure</code>	是否开启红外自动曝光

需要注意：

`--show-args` 显示的是启动文件作者设置的默认值，不代表连接的每一种相机都支持这些默认值。

相机真正支持什么格式，必须以运行时打印的 `Available profiles` 为准。

28. 第一次启动红外失败：Y8格式不匹配

第一次使用了：

```
ros2 launch orbbec_camera ob_camera.launch.py \  
  enable_color:=false \  
  enable_depth:=false \  
  enable_ir:=true \  
  ir_width:=640 \  
  ir_height:=400 \  
  ir_fps:=30 \  
  ir_format:=Y8 \  
  enable_point_cloud:=false
```

驱动随后报错：

```
Failed to get ir profile: Invalid input  
No matched video stream profile found!
```

并明确指出请求的配置是：

```
Width: 640  
Height: 400  
FPS: 30  
Format: OB_FORMAT_Y8
```

这句话的意思是：

驱动要求相机提供 `640x400@30 FPS Y8` 红外流，但是相机支持的模式列表中找到完全一致的配置。

因此相机节点退出：

```
Because can not set this stream, so exit
process has died
exit code 255
```

这不是硬件损坏、USB权限错误或ROS 2安装错误，而是数据格式不匹配。

29. 相机实际支持的红外模式

驱动失败后列出了 Gemini Max 实际支持的红外模式，包括：

```
640x400 30fps Y10
320x200 5fps Y10
320x200 10fps Y10
320x200 15fps Y10
320x200 30fps Y10
320x200 60fps Y10
640x400 5fps Y10
640x400 10fps Y10
640x400 15fps Y10
640x400 60fps Y10
1280x800 15fps Y10
1280x800 30fps Y10
```

这些模式说明：

- 1. 这台 Gemini Max 的红外格式是 Y10
- 2. 它不接受启动文件默认的 Y8
- 3. 640x400@30 FPS Y10 是明确支持的配置
- 4. 最高分辨率可以达到 1280x800
- 5. 部分低分辨率模式可以达到 60 FPS

本次选择：

```
640x400@30 FPS Y10
```

因为它在分辨率、帧率和 Jetson 处理压力之间较为平衡，也与已经跑通的深度图分辨率一致。

30. Y8、Y10和Y12是什么

这些名称描述每个像素数据的有效位数。

格式	有效位数	理论数值范围	当前用途
Y8	8位	0 ~ 255	本相机当前红外模式不支持
Y10	10位	0 ~ 1023	Gemini Max红外图
Y12	12位	0 ~ 4095	Gemini Max深度数据格式

红外图中的像素值表示接收到的红外光强度：



数值较小 → 接收到的红外光较弱

数值较大 → 接收到的红外光较强

深度图中的值则主要表示距离信息。因此：



红外Y10：光强数据

深度Y12：距离测量数据

两者虽然都以灰度图形式显示，但物理意义完全不同。

31. 使用Y10成功启动红外流

将格式修改为 **Y10** 后，使用：



```
ros2 launch orbbec_camera ob_camera.launch.py \  
  enable_color:=false \  
  enable_depth:=false \  
  enable_ir:=true \  
  ir_width:=640 \  
  ir_height:=400 \  
  ir_fps:=30 \  
  ir_format:=Y10 \  
  enable_point_cloud:=false
```

日志出现：



```
stream ir is enabled  
width: 640  
height: 400  
fps: 30  
Format: OB_FORMAT_Y10
```

并继续显示：



```
Enable ir stream  
Stream ir width: 640 height: 400 fps: 30 format: Y10  
Device SV1301S_U3 connected
```

这些日志已经证明：



相机被识别

→ 驱动找到了匹配的Y10红外模式

→ 红外数据流成功建立

→ 相机开始向ROS 2节点传输红外帧

设备信息仍然一致：



设备: SV1301S_U3
序列号: AY27552006P
固件: RD3013
Wrapper: 1.5.22
SDK: 1.10.37
USB连接: USB 2.0

32. 红外ROS 2话题验证

执行:



```
ros2 topic list | grep -E "ir|infra"
```

得到:



```
/camera/ir/camera_info  
/camera/ir/image_raw
```

完整话题列表中包含:



```
/camera/depth_filter_status  
/camera/ir/camera_info  
/camera/ir/image_raw  
/parameter_events  
/rosout  
/tf  
/tf_static
```

这与当前启动参数完全对应。

由于本次关闭了:



```
enable_depth:=false  
enable_color:=false
```

所以没有深度图和彩色图话题是正常现象。

/camera/ir/image_raw

这是实际的红外图像数据, 消息类型一般为:



```
sensor_msgs/msg/Image
```

/camera/ir/camera_info

这是红外相机的标定信息, 包括:

- 图像尺寸
- 焦距
- 光心
- 畸变参数
- 相机投影矩阵

后续进行红外图像校正、空间投影或红外与深度配准时会用到这些参数。

33. 使用rqt_image_view查看红外图

执行：



```
ros2 run rqt_image_view rqt_image_view
```

选择：



```
/camera/ir/image_raw
```

rqt_image_view 已经能够订阅该话题，说明以下通信链路成立：



```
Gemini Max红外传感器  
→ Orbbec SDK  
→ ROS 2相机节点  
→ /camera/ir/image_raw  
→ rqt_image_view
```

截图中的红外图像整体较黑，但这和“没有红外数据”不是同一个问题。

可能原因包括：

1. 当前场景反射的红外光较弱
2. 相机自动曝光尚未稳定
3. **Y10** 是10位数据，而显示工具的灰度映射范围不理想
4. MobaXterm远程显示造成暗部细节不明显
5. 被拍摄物体距离或材质不适合反射红外光

可以把手或具有明显轮廓的物体放在相机前方约 **0.5~1.5 m** 处，并等待几秒观察画面变化。

红外光不可见，不要因为肉眼看不到补光就认为红外发射器没有工作，也不要近距离长时间直视红外发射窗口。

34. 建议补充验证红外帧率

为了像深度图和点云一样客观验证红外流，应执行：



```
ros2 topic hz /camera/ir/image_raw
```

预期结果接近：



average rate: 30

如果接近30 Hz，则可以确认：



配置帧率：30 FPS
实际ROS 2发布频率：约30 Hz

还可以检查消息基本信息：



```
ros2 topic info /camera/ir/image_raw
```

它可以确认：

- 消息类型
- 发布者数量
- 订阅者数量

35. 命令末尾多出波浪号的问题

截图中的实际命令末尾写成了：



```
enable_point_cloud:=false~
```

正确写法应当是：



```
enable_point_cloud:=false
```

多出来的 `~` 导致 `false~` 被当成字符串，而不是布尔值 `false`。

因此日志出现：



```
Wrong parameter type  
parameter {enable_point_cloud} is of type {bool}  
setting it to {string} is not allowed
```

这句话的含义是：



驱动要求布尔值：false
实际收到字符串："false~"
类型不匹配，因此拒绝设置

本次红外仍然成功，是因为：

- 深度流已经明确关闭
- 没有深度图就不会生成有效点云
- 这个错误没有阻止红外数据流启动

但后续命令必须删除末尾的 `~`，否则驱动可能继续使用该参数的默认值。

36. `^[[200~source` 错误是什么

截图中还出现了：

```
● ● ●
^[[200~source /opt/ros/humble/setup.bash
-bash: $'\E[200~source': command not found
```

其中的：

```
● ● ●
^[[200~
```

是终端“括号粘贴模式”的控制字符。正常情况下它应该由终端自动处理，不应该作为命令内容发送给 Bash。

这通常是 MobaXterm 粘贴时偶发的终端兼容问题，不是 ROS 问题。

解决办法：

1. 按 `Ctrl+C` 清空当前输入
2. 手动重新输入命令
3. 或者一次只粘贴一行

正确命令仍然是：

```
● ● ●
source /opt/ros/humble/setup.bash
```

37. 命令行中的 `>` 是什么意思

输入多行命令时，终端出现：

```
● ● ●
>
```

不是错误。

它表示上一行以反斜杠结尾：

```
● ● ●
\
```

Bash 认为命令还没有写完，因此等待输入下一行。

例如：

```
● ● ●
ros2 launch orbbec_camera ob_camera.launch.py \
  enable_ir:=true \
  ir_format:=Y10
```

只有输入最后一行并按回车后，整条命令才会执行。

需要注意：反斜杠 `\` 必须是这一行的最后一个有效字符，后面不要再添加其他字符。

38. RViz退出时的Segmentation fault

截图中之前的 RViz 在按下 `Ctrl+C` 后出现：

```
signal_handler(signum=2)
Segmentation fault (core dumped)
```

其中：

```
signum=2
```

对应用户按下 `Ctrl+C` 产生的中断信号。

Segmentation fault 表示 RViz 在退出和释放图形资源时发生了非法内存访问。考虑到当时：

- RViz通过MobaXterm X11运行
- 点云渲染负载较高
- 消息队列持续满载
- 崩溃发生在退出阶段
- 相机驱动之后仍能正常启动

因此这次崩溃更可能属于 RViz/X11 的退出问题，不代表 Gemini Max 或ROS 2驱动损坏。

如果只在退出时偶尔发生，可以暂时记录但不需要重新安装。若每次启动RViz都立即崩溃，才需要进一步检查OpenGL渲染环境和内存占用。

39. 红外自动曝光和LDP

参数列表中显示：

```
enable_ir_auto_exposure: true
```

表示红外相机默认启用自动曝光。驱动会根据当前场景亮度调整曝光时间，使红外画面不过亮或过暗。

日志还显示：

```
Setting LDP to ON
```

LDP 与相机的红外投射或激光保护控制有关。它属于相机底层功能，不等于热成像，也不需要为了普通红外图显示而手动修改。

现阶段建议保持：

```
enable_ir_auto_exposure:=true
```

不要过早手动调曝光，因为自动曝光更适合先验证设备是否正常工作。

40. 当前三个视觉输出的完成状态

功能	ROS 2话题	格式	配置帧率	当前状态
深度图	/camera/depth/image_raw	Y12	30 FPS	已跑通
三维点云	/camera/depth/points	PointCloud2	约30 Hz	已跑通
红外图	/camera/ir/image_raw	Y10	30 FPS	已跑通
彩色图	尚未验证	默认MJPG	未验证	暂不启用

这里需要区分：

- 深度图：每个像素表示距离
- 红外图：每个像素表示红外反射强度
- 点云：把深度像素转换为三维坐标

41. 单独启动红外的标准命令

后续单独研究红外图时，使用：

```
source /opt/ros/humble/setup.bash
source ~/orbbeo_ros2_ws/install/setup.bash

ros2 launch orbbeo_camera ob_camera.launch.py \
  enable_color:=false \
  enable_depth:=false \
  enable_ir:=true \
  ir_width:=640 \
  ir_height:=400 \
  ir_fps:=30 \
  ir_format:=Y10 \
  enable_point_cloud:=false
```

查看红外图：

```
ros2 run rqt_image_view rqt_image_view
```

选择：

```
/camera/ir/image_raw
```

检查帧率：

```
ros2 topic hz /camera/ir/image_raw
```

42. 同时启动深度、点云和红外

在红外单独运行成功的基础上，可以测试组合启动：

```
● ● ●
source /opt/ros/humble/setup.bash
source ~/orbbeo_ros2_ws/install/setup.bash

ros2 launch orbbeo_camera ob_camera.launch.py \
  enable_color:=false \
  enable_depth:=true \
  depth_width:=640 \
  depth_height:=400 \
  depth_fps:=30 \
  depth_format:=Y12 \
  enable_ir:=true \
  ir_width:=640 \
  ir_height:=400 \
  ir_fps:=30 \
  ir_format:=Y10 \
  enable_point_cloud:=true
```

然后检查：

```
● ● ●
ros2 topic list | grep -E "depth|points|ir"
```

理论上应包含：

```
● ● ●
/camera/depth/camera_info
/camera/depth/image_raw
/camera/depth/points
/camera/ir/camera_info
/camera/ir/image_raw
```

需要说明：当前截图证明的是“红外单独运行成功”。深度、点云和红外同时开启是否稳定，还应通过组合启动和话题帧率测量验证。

由于相机当前工作在 **USB 2.0**，如果组合启动失败，应先保留深度30 FPS，将红外降低为相机明确支持的：

```
● ● ●
640x400@15 FPS Y10
```

对应参数：

```
● ● ●
ir_fps:=15
```

不要随意填写相机没有列出的分辨率、帧率或格式组合。

43. 本阶段最重要的调试经验

本阶段形成了一个通用的视觉设备调试方法：



- 先读取启动参数
 - 用最少的数据流单独启动
 - 查看错误日志
 - 找到设备实际支持的profile
 - 按完整profile修改参数
 - 检查ROS话题
 - 查看图像
 - 测量真实帧率
 - 最后再组合多个数据流

其中最关键的原则是：

启动文件默认值不等于硬件支持值，设备运行时列出的可用Profile才是最终依据。

本次具体表现为：



启动文件默认红外格式：Y8
Gemini Max实际红外格式：Y10

修改为 Y10 后，红外数据流成功运行。

至此，已经完成 Gemini Max 的三项核心感知能力：



二维距离感知：深度图
三维空间感知：点云
红外光强感知：红外图

下一阶段可以进入真正的计算机视觉任务，例如读取指定像素距离、深度图伪彩色显示、点云降采样、地面分割、障碍物检测，以及将相机坐标转换到机器人 `base_link` 坐标系。